```
#pragma once

// Debug Macros, for printing to the Serial Monitor
// only when 'DEBUG_MODE' is defined

#include <stdint.h>

#define MSG_TERMINATOR '^'

#define ReqActBox "RAB"
#define UpdBoxes "UPD"
#define ReqInvCount "IVC"
#define ReqInvWin "IVW"

#define INVENTORY_FILE "/inventory.csv"

#define PROTO_VERSION 0x01

// Debug defines, made such that when debug mode is off (and the system is in
// production mode) various types of processes strictly for debugging purposes
// are turned off such that the system can focus on main tasks.
#define DEBUG_MODE

#ifdef DEBUG_MODE

  #ifdef NODEMCU_FILE
    #define d_SerialPrint(x) Serial.print(x)
    #define d_SerialPrintln(x) Serial.println(x)
  #else
    #define d_SerialPrint(x) Serial.print(F(x))
    #define d_SerialPrintln(x) Serial.println(F(x))
  #endif
  #define d_SerialBegin(x) Serial.begin(x)
  #define d_SerialPrintV(x) Serial.print(x)     // Print Value
  #define d_SerialPrintlnV(x) Serial.println(x) // Print Value
  #define d_SerialPrintf(...) Serial.printf(__VA_ARGS__)
  #define d_SerialPrintGroup(...) Serial.print(__VA_ARGS__)
  #define d_SerialPrintGroupln(...) Serial.println(__VA_ARGS__)
  #define d_delay(x) delay(x)

#else

  #define d_SerialBegin(x)
  #define d_SerialPrint(x)
  #define d_SerialPrintV(x)
  #define d_SerialPrintln(x)
  #define d_SerialPrintlnV(x)
  #define d_SerialPrintf(...)
  #define d_SerialPrintGroup(...)
```

```
  #define d_SerialPrintGroupln(...)
  #define d_delay(x)

#endif

// Eponymous.
enum FoodCondition : uint8_t
{
  FRESH = 0,
  GOING_BAD = 1,
  EXPIRED = 2,
  UNKNOWN = 3,
};

// Struct type to easily pack value types.
struct ThresholdValues
{
  uint16_t alc_bad;
  uint16_t alc_exp;
  uint16_t ch3_bad;
  uint16_t ch3_exp;
};

enum ProfileID : uint8_t
{
  PROFILE_UNKNOWN = 0,
  FRUIT_FAST = 1,
  FRUIT_MEDIUM = 2,
  FRUIT_SLOW = 3,
  EXPERIMENTAL = 4,
  TIME_ONLY = 5
};

// To turn a sensor 'off', simply set the threshold value to 9999.
const ThresholdValues profile_table[] = {
    {9999, 9999, 9999, 9999}, // UNKNOWN
    {180, 320, 9999, 9999},   // FRUIT_FAST
    {220, 380, 9999, 9999},   // FRUIT_MEDIUM
    {260, 440, 9999, 9999},   // FRUIT_SLOW
    {220, 380, 500, 700},     // EXPERIMENTAL
    {9999, 9999, 9999, 9999}  // TIME_ONLY
};

// Wifi Password Pair Type
typedef struct
{
  char ssid[20];
  char pass[20];
} wifi_pass_t;
```

```
// Boxes Type
struct box_t
{
  char index[3];
  volatile FoodCondition State;
  volatile ProfileID p_id;
  uint64_t address;
};

typedef enum
{
  BoxInfo,
  ActiveBoxes,
  UpdateBoxes
} Requests;

struct Packet
{
  uint8_t header;
  uint8_t payload;
  uint8_t crc;
};

enum PacketType : uint8_t
{
  PKT_REQUEST = 0,
  PKT_RESPONSE = 1,
  PKT_CONFIG = 2
};

inline constexpr uint64_t CalcAddress(const char *index)
{
  return (0xF0F0F0F000LL | (uint64_t)index[0]) | ((uint64_t)index[1] << 8);
}

inline uint8_t MakeHeader(uint8_t type, uint8_t sequence)
{
  return ((PROTO_VERSION & 0x01) << 7) | ((type & 0x03) << 5) | (sequence & 0x0F);
}

inline uint8_t MakePayload(FoodCondition condition, uint8_t additional_data = 0)
{
  return ((condition & 0x03) << 6) | (additional_data & 0x3F);
}

inline uint8_t ComputeCRC(uint8_t header, uint8_t payload)
{
  return header ^ payload;
```

```
}

// Packs data into a valid 3-byte packet.
inline void PackPacket(Packet &packet, uint8_t type, uint8_t sequence,
FoodCondition condition,
                       uint8_t additional_data = 0)
{
  packet.header = MakeHeader(type, sequence);
  packet.payload = MakePayload(condition, additional_data);
  packet.crc = ComputeCRC(packet.header, packet.payload);
}

// Unpacks incoming packet data.
inline bool UnpackPacket(const Packet &packet, uint8_t &type, uint8_t &sequence,
FoodCondition &condition,
                         uint8_t &additional_data)
{
  if (ComputeCRC(packet.header, packet.payload) != packet.crc)
    return false;

  uint8_t version = (packet.header >> 7) & 0x01;
  if (version != PROTO_VERSION)
    return false;

  type = (packet.header >> 5) & 0x03;
  sequence = packet.header & 0x0F;

  condition = (FoodCondition)((packet.payload >> 6) & 0x03);
  additional_data = packet.payload & 0x3F;

  return true;
}
```

```cpp
// main.cpp

#define POLL_INTERVAL_MS 5000
#define REFRESH_INT_MS 30000

#include "box_actions.h"
#include "interop_comms.h"
#include <.common/comm.h>
#include <Arduino.h>
#include <SoftwareSerial.h>

unsigned long last_poll = 0;
unsigned long last_refresh = 0;

unsigned long time = 0;
// const int SENSOR_READ_DELAY = 100;
const int status_pins[] = {Pins::FRESH_PIN, Pins::GOING_BAD_PIN, Pins::EXPIRED_PIN};

int DetermineFoodStatus();
void UpdateStatusDisplay(int Cond);

SoftwareSerial nodemcu(5, 6);

void setup()
{
  d_SerialBegin(9600);

  RadioSetup();

  pinMode(Pins::FRESH_PIN, OUTPUT);
  pinMode(Pins::GOING_BAD_PIN, OUTPUT);
  pinMode(Pins::EXPIRED_PIN, OUTPUT);

  pinMode(Pins::ALC_SENSOR_PIN, INPUT);
  pinMode(Pins::CH3_SENSOR_PIN, INPUT);

  digitalWrite(Pins::FRESH_PIN, HIGH);
  delay(1000);
  digitalWrite(Pins::FRESH_PIN, LOW);
  delay(1000);
  digitalWrite(Pins::GOING_BAD_PIN, HIGH);
  delay(1000);
  digitalWrite(Pins::GOING_BAD_PIN, LOW);
  delay(1000);
  digitalWrite(Pins::EXPIRED_PIN, HIGH);
  delay(1000);
  digitalWrite(Pins::EXPIRED_PIN, LOW);

  d_SerialPrintln("SFES Main Indicator Init'sed");

  nodemcu.begin(9600);
```

```
  char response[120];

  d_SerialPrintln("Entering Setup, sending Active Boxes request.");
  if (NodeMCUTransmit(Requests::ActiveBoxes, &nodemcu, NULL, response,
sizeof(response)))
  {
    d_SerialPrint("Receieved input:");
    d_SerialPrintlnV(response);
    ParseBoxes(response);
  }
  else
  {
    d_SerialPrintln("Failed to get active boxes");
  }
}

void loop()
{
  unsigned long now = millis();

  if (now - last_poll >= POLL_INTERVAL_MS)
  {
    d_SerialPrintln("Polling...");
    last_poll = now;
    Poller();
    NodeMcuUpdate(&nodemcu);
  }
  if (now - last_refresh >= REFRESH_INT_MS)
  {
    d_SerialPrintln("Refreshing...");
    last_refresh = now;
    char response[120];

    if (NodeMCUTransmit(Requests::ActiveBoxes, &nodemcu, NULL, response,
sizeof(response)))
    {
      d_SerialPrint("Receieved input:");
      d_SerialPrintlnV(response);
      ParseBoxes(response);
    }
    else
      d_SerialPrintln("Failed to get active boxes");
  }
}
```

```
// box_actions.h

#include <.common/comm.h>
#include <Arduino.h>
#include <SoftwareSerial.h>

#define MAX_BOXES 2
#define TIMEOUT_MS 500
#define TIME_TO_WAIT_FOR_RESPONSE 250

enum Pins
{
  FRESH_PIN = 2,
  GOING_BAD_PIN = 3,
  EXPIRED_PIN = 4,
  ALC_SENSOR_PIN = A0,
  CH3_SENSOR_PIN = A1
};

enum ThresholdPins // Current Testing Values
{
  ALC_THRESHOLD_BAD = 160,
  ALC_THRESHOLD_EXP = 500,
  CH3_THRESHOLD_BAD = 400,
  CH3_THRESHOLD_EXP = 600
};

FoodCondition evalLocal();
void RadioSetup();
void ParseBoxes(char *resp);

void Poller();

void NodeMcuUpdate(SoftwareSerial *sender);

// box_actions.cpp

#include "box_actions.h"

#include ".common/comm.h"

#include "interop_comms.h"
#include <Arduino.h>
#include <RF24.h>
#include <SoftwareSerial.h>
#include <stdint.h>

constexpr uint64_t MAIN_ADDRESS = 0xF0F0F0AA11LL;

RF24 radio(9, 10);

box_t active_boxes[MAX_BOXES];
uint8_t active_box_count = 0;
```

```
ThresholdValues thresholds;

inline void PrintAddress(uint64_t addr)
{
  d_SerialPrint(" 0x");

  for (int i = 4; i >= 0; i--) // nRF24 uses 5-byte addresses
  {
    uint8_t byte = (addr >> (8 * i)) & 0xFF;

    if (byte < 0x10)
      d_SerialPrintV('0');

    d_SerialPrintGroup(byte, HEX);
  }
  d_SerialPrintln("");
}
```

```
FoodCondition EvalLocal()
{
  int alc_reading = analogRead(Pins::ALC_SENSOR_PIN);
  int ch3_reading = analogRead(Pins::CH3_SENSOR_PIN);

  d_SerialPrint("Alcohol Values: ");
  d_SerialPrintlnV(alc_reading);
  d_SerialPrint("CH3 Values : ");
  d_SerialPrintlnV(ch3_reading);

  if (alc_reading >= thresholds.alc_exp || ch3_reading >= thresholds.ch3_exp)
  {
    d_SerialPrintln("\nExpired!\n\n");
    digitalWrite(Pins::EXPIRED_PIN, HIGH);
    digitalWrite(Pins::FRESH_PIN, LOW);
    digitalWrite(Pins::GOING_BAD_PIN, LOW);
    d_SerialPrint("Current Threshold Value is:\nAlc:");
    d_SerialPrintlnV(thresholds.alc_exp);
    d_SerialPrint("ch3:");
    d_SerialPrintlnV(thresholds.ch3_exp);
    return FoodCondition::EXPIRED;
  }
  else if (alc_reading >= thresholds.alc_bad || ch3_reading >= thresholds.ch3_bad)
  {
    d_SerialPrintln("\nGoing Bad!\n\n");
    digitalWrite(Pins::GOING_BAD_PIN, HIGH);
    digitalWrite(Pins::FRESH_PIN, LOW);
    digitalWrite(Pins::EXPIRED_PIN, LOW);
    d_SerialPrint("Current Threshold Value is:\nAlc:");
    d_SerialPrintlnV(thresholds.alc_bad);
    d_SerialPrint("ch3:");
    d_SerialPrintlnV(thresholds.ch3_bad);
    return FoodCondition::GOING_BAD;
  }
  else
```

```
  {
    d_SerialPrintln("\nFresh!\n\n");
    digitalWrite(Pins::FRESH_PIN, HIGH);
    digitalWrite(Pins::GOING_BAD_PIN, LOW);
    digitalWrite(Pins::EXPIRED_PIN, LOW);
    return FoodCondition::FRESH;
  }
}

void RadioSetup()
{
  radio.begin();
  radio.setPALevel(RF24_PA_LOW);
  radio.setDataRate(RF24_250KBPS);
  radio.setChannel(76);
  radio.setRetries(5, 15);
  radio.openReadingPipe(1, MAIN_ADDRESS);
  radio.startListening();

  d_SerialPrintGroup(radio.isChipConnected() ? "Radio initialized (UNO)" : " Radio
Uninitialized! Restart");
  d_SerialPrint("Listening on: ");
  PrintAddress(MAIN_ADDRESS);
  radio.stopListening();
}

void ParseBoxes(char *resp)
{
  memset(active_boxes, 0, sizeof(active_boxes));
  active_box_count = 0;

  char *token = strtok(resp, ";");

  while (token != NULL && active_box_count < MAX_BOXES)
  {
    while (*token == ' ')
      token++;

    size_t len = strlen(token);
    while (len > 0 && (token[len - 1] == ' ' || token[len - 1] == '\r' || token[len - 1]
== '\n'))
    {
      token[len - 1] = '\0';
      len--;
    }

    if (len >= 4) // Like, the input must atleast be something like "A0,1;", which is 4
[5]characters
    {
      char *comma = strchr(token, ',');
      if (comma != NULL)
      {
```

```
*comma = '\0';
char *valueStr = comma + 1;

uint8_t value = (uint8_t)atoi(valueStr);

// fill struct
active_boxes[active_box_count].index[0] = token[0];
active_boxes[active_box_count].index[1] = token[1];
active_boxes[active_box_count].index[2] = '\0';

// set state AFTER p_id is known
if (token[0] == 'A' && token[1] == '0')
{
  thresholds = profile_table[value];
  active_boxes[active_box_count].State = EvalLocal();
}
else
        {
          active_boxes[active_box_count].State = FoodCondition::UNKNOWN;
          active_boxes[active_box_count].address =
CalcAddress(active_boxes[active_box_count].index);

          if (active_boxes[active_box_count].p_id != value) // Only update vals when
it's actually changed
          {
            active_boxes[active_box_count].p_id = (ProfileID)value;
            d_SerialPrint("Profile ID: ");
            d_SerialPrintlnV(active_boxes[active_box_count].p_id);

            Packet config;
            PackPacket(config, PacketType::PKT_CONFIG, 9, FoodCondition::UNKNOWN,
active_boxes[active_box_count].p_id);
            radio.openWritingPipe(active_boxes[active_box_count].address);

            radio.stopListening();

            radio.openWritingPipe(active_boxes[active_box_count].address);
            bool ok = radio.write(&config, sizeof(config));

            if (!ok)
              d_SerialPrintln("Config upload failed");
          }
        }

        active_box_count++;
      }
    }

    token = strtok(NULL, ";");
}

d_SerialPrint("Parsed ");
d_SerialPrintV(active_box_count);
```

```
  d_SerialPrintln(" boxes");

  for (int i = 0; i < active_box_count; i++)
  {
    d_SerialPrint("Box ");
    d_SerialPrintV(i);
    d_SerialPrint(": ");
    d_SerialPrintlnV(active_boxes[i].index);
  }
}

void ClearRadioRX()
{
  while (radio.available())
  {
    Packet junk;
    radio.read(&junk, sizeof(junk));
  }
}

void Poller()
{
  static int current_box = 0;
  static uint8_t seq = 0;

  if (active_box_count == 0)
    return;

  if (current_box >= active_box_count)
    current_box = 0;

  int start_box = current_box;

  while (strcmp(active_boxes[current_box].index, "A0") == 0)
  {
    current_box++;

    if (current_box >= active_box_count)
      current_box = 0;

    if (current_box == start_box) // If only A0 exists
      return;
  }

  uint8_t my_seq = seq;

  Packet req;
  PackPacket(req, PacketType::PKT_REQUEST, my_seq, FoodCondition::UNKNOWN,
active_boxes[current_box].p_id);

  d_SerialPrint("Polling box: ");
  d_SerialPrintlnV(active_boxes[current_box].index);
```

```
d_SerialPrint("Polling address: ");
PrintAddress(active_boxes[current_box].address);

ClearRadioRX();
radio.stopListening();
radio.openWritingPipe(active_boxes[current_box].address);

bool ok = radio.write(&req, sizeof(req));

d_SerialPrint("Poller write = ");
d_SerialPrintlnV(ok ? "OK" : "FAIL");

radio.openReadingPipe(1, MAIN_ADDRESS);
radio.startListening();

unsigned long start = millis();
bool got_matching_response = false;

while (millis() - start < TIME_TO_WAIT_FOR_RESPONSE)
{
  if (!radio.available())
    continue;

  Packet resp;
  radio.read(&resp, sizeof(resp));

  uint8_t type, rseq;
  FoodCondition cond;
  uint8_t flags;

  if (!UnpackPacket(resp, type, rseq, cond, flags))
  {
    d_SerialPrintln("Poller: bad response packet");
    continue;
  }

if (type != PKT_RESPONSE)
{
  d_SerialPrintln("Silly Nano! You have sent the wrong type of request.");
  continue;
}

if (rseq != my_seq)
{
  d_SerialPrint("Poller: seq mismatch, got ");
  d_SerialPrintV(rseq);
  d_SerialPrint(" expected ");
  d_SerialPrintlnV(my_seq);
  continue;
}

d_SerialPrintln("Poller: valid response");
active_boxes[current_box].State = cond;
```

```
  got_matching_response = true;
  break;
}

if (!got_matching_response)
{
  d_SerialPrintln("Poller: response timeout");
  active_boxes[current_box].State = FoodCondition::UNKNOWN;
}

radio.stopListening();
radio.openReadingPipe(1, MAIN_ADDRESS);
radio.startListening();

current_box++;
  if (current_box >= active_box_count)
    current_box = 0;

  seq = (seq + 1) & 0x0F;
}

void NodeMcuUpdate(SoftwareSerial *sender)
{
  char csv_data[120];
  size_t idx = 0;

  for (int i = 0; i < active_box_count; i++)
  {
    const char *index = active_boxes[i].index;
    uint8_t state = (uint8_t)active_boxes[i].State;

    int written = snprintf(csv_data + idx, sizeof(csv_data) - idx, "%s,%d\n", index,
state);
  if (written < 0)
    break;

  idx += (size_t)written;

  if (idx >= sizeof(csv_data))
  {
    idx = sizeof(csv_data) - 1;
    break;
  }
}

csv_data[idx] = '\0';

char dummy[16] = {0};

if (!NodeMCUTransmit(Requests::UpdateBoxes, sender, csv_data, dummy, sizeof(dummy)))
```

```
    d_SerialPrintln("Update failed");
}
```

```
// interop_comms.h

#include <Arduino.h>
#include <SoftwareSerial.h>

#include ".common/comm.h"

bool NodeMCUTransmit(Requests request, SoftwareSerial *serial, const char *payload,
char *out_buffer,
                     size_t buffer_size);

// interop_comms.cpp

#include <Arduino.h>
#include <SoftwareSerial.h>

#include ".common/comm.h"
#include "interop_comms.h"

bool NodeMCUTransmit(Requests request, SoftwareSerial *serial, const char *payload,
char *out_buffer,
                     size_t buffer_size)
{
  size_t idx = 0;
  switch (request)
  {
    case Requests::ActiveBoxes:
      serial->print(ReqActBox);
      break;
    case Requests::UpdateBoxes:
      serial->print(UpdBoxes);
      break;
    default:
      return false;
  }
  serial->print(MSG_TERMINATOR);
  serial->flush();
  unsigned long start = millis();
  while (millis() - start < 1000)
  {
    while (serial->available())
    {
      char c = serial->read();
      if (c == MSG_TERMINATOR)
      {
        out_buffer[idx] = '\0';
        goto RESPONSE_DONE;
      }
      if (idx < buffer_size - 1)
        out_buffer[idx++] = c;
      else
        idx = 0;
    }
```

```
  }
  return false;
RESPONSE_DONE:
  if (request == Requests::UpdateBoxes)
  {
    if (strcmp(out_buffer, "OK") != 0)
    {
      d_SerialPrintln("NodeMCU did not ACK update");
      return false;
    }
    if (payload != NULL)
    {
      serial->print(payload);
      serial->print(MSG_TERMINATOR);
      serial->flush();
    }
    return true;
  }
  return true;
}
```